

PRUEBAS DEL SOFTWARE: USO DE UN FRAMEWORK DE TESTING

TRABAJO PRÁCTICO 1

Integrantes:

Rodriguez Rodriguez Andy Alejandro

Mamani Victor Daniel

Michelle Camille Deaver

Producto.cs

```
public class Producto : IProducto
{
    2 references
    string nombre = "";
    2 references
    decimal precio;
    2 references
    string? categoria;

    8 references
    public string Nombre { get => nombre; set => nombre = value; }
    8 references
    public decimal Precio { get => precio; set => precio = value; }
    2 references
    public string? Categoria { get => categoria; set => categoria = value; }

    11 references
    public Producto(string nombre, decimal precio, string categoria)
    {
        Nombre = nombre;
        Precio = precio;
        Categoria = categoria;
    }
}
```

Tienda.cs

```
public class Tienda
{
    5 references
    List<Producto> inventario = [];

    7 references
    public Tienda()
    {
        inventario = new List<Producto>();
    }

    9 references
    public void AgregarProducto(Producto producto)
    {
        inventario.Add(producto);
    }

    9 references
    public Producto BuscarProducto(string nombre)
    {
        Producto? producto = inventario.FirstOrDefault(p => p.Nombre == nombre);

        return producto;
    }

    3 references
    public bool EliminarProducto(string nombre)
    {
        Producto? producto = BuscarProducto(nombre);

        if (producto != null)
        {
            inventario.Remove(producto);
            return true;
        }
        return false;
    }

    2 references
    public void MostrarInventario()
    {
        foreach (var producto in inventario)
        {
            Console.WriteLine(producto.Nombre);
        }
    }
}
```

1.

- Xunit es un framework de pruebas unitarias para .NET (C#, F# y otros). Permite escribir, ejecutar y automatizar tests para verificar el comportamiento del código.
- Características clave:
 - Usa atributos como [Fact] y [Theory] para definir pruebas.
 - Se integra con dotnet test y Visual Studio Code.

ProductoTest.cs

```
using Xunit;

0 references
public class ProductoTests
{
    [Fact]
    0 references
    public void Producto_Creacion_Correcta()
    {
        var producto = new Producto("Leche", 150, "Lácteos");
        Assert.Equal("Leche", producto.Nombre);
        Assert.Equal(150, producto.Precio);
        Assert.Equal("Lácteos", producto.Categoria);
    }
}
```

TiendaTest.cs

```
using Xunit;

0 references
public class TiendaTests
{
    [Fact]
    0 references
    public void AgregarProducto_ProductoSeAgrega()
    {
        var tienda = new Tienda();
        var producto = new Producto("Pan", 50, "Alimentos");

        tienda.AgregarProducto(producto);

        var buscado = tienda.BuscarProducto("Pan");

        Assert.NotNull(buscado);
        Assert.Equal("Pan", buscado.Nombre);
    }

    [Fact]
    0 references
    public void EliminarProducto_ProductoExistente_SeElimina()
    {
        var tienda = new Tienda();
        var producto = new Producto("Jugo", 120, "Bebidas");

        tienda.AgregarProducto(producto);
        tienda.EliminarProducto("Jugo");

        Assert.Throws<KeyNotFoundException>(
            () => tienda.BuscarProducto("Jugo"));
    }
}
```

- Pruebas de unidad:
 - **Producto_Creacion_Correcta:** verifica de forma independiente que un objeto Producto se cree correctamente con su nombre, precio y categoría.
 - **AgregarProducto_ProductoSeAgrega:** verifica que Tienda pueda agregar correctamente un producto al inventario.
 - **EliminarProducto_ProductoExistente_SeElimina:** verifica que un producto existente pueda eliminarse correctamente de la tienda. Las pruebas de búsqueda y eliminación de productos no existentes se encuentran en **TiendaExcepcionesTests.cs**, donde se verifica que se lance la excepción correspondiente.
- Pruebas de integración:
 - **AgregarProducto_ProductoSeAgrega:** combinan interacción de Tienda y Producto, verificando que el flujo completo funciona.

2.

Actualización en Producto.cs

```
public void ActualizarPrecio(decimal nuevoPrecio)
{
    if (nuevoPrecio < 0)
        throw new ArgumentException("El precio no puede ser negativo.");

    Precio = nuevoPrecio;
}
```

Actualización en Tienda.cs

```
public Producto BuscarProducto(string nombre)
{
    var producto = inventario.FirstOrDefault(p => p.Nombre == nombre);
    if (producto == null)
        throw new KeyNotFoundException($"Producto '{nombre}' no encontrado.");
    return producto;
}

3 references
public void EliminarProducto(string nombre)
{
    var producto = inventario.FirstOrDefault(p => p.Nombre == nombre);
    if (producto == null)
        throw new KeyNotFoundException($"No se puede eliminar '{nombre}', no existe.");
    inventario.Remove(producto);
}
```

ProductoExcepcionesTests.cs

```
using Xunit;

0 references
public class ProductoExcepcionesTests
{
    [Fact]
    0 references
    public void ActualizarPrecio_Negativo_LanzaExcepcion()
    {
        var producto = new Producto("Leche", 100, "Lácteos");
        Assert.Throws<ArgumentException>(() => producto.ActualizarPrecio(-50));
    }
}
```

TiendaExcepcionesTests.cs

```
using Xunit;

0 references
public class TiendaExcepcionesTests
{
    [Fact]
    0 references
    public void BuscarProducto_NoExistente_LanzaExcepcion()
    {
        var tienda = new Tienda();
        Assert.Throws<KeyNotFoundException>(() => tienda.BuscarProducto("Queso"));
    }

    [Fact]
    0 references
    public void EliminarProducto_NoExistente_LanzaExcepcion()
    {
        var tienda = new Tienda();
        Assert.Throws<KeyNotFoundException>(() => tienda.EliminarProducto("Agua"));
    }
}
```

- Sí se puede escribir primero los tests: se escribe un test que indique el comportamiento esperado, se ejecuta el test y falla (porque aún no hay implementación), luego se modifica el código para que pase el test, y se refactoriza si es necesario, manteniendo los tests verdes.

3.

Modificación en Tienda.cs

```
public void AplicarDescuento(IPProducto producto, decimal porcentaje)
{
    if (producto == null) throw new ArgumentNullException(nameof(producto));
    decimal nuevoPrecio = producto.Precio * (1 - porcentaje / 100);
    producto.ActualizarPrecio(nuevoPrecio);
}
```

IPProducto.cs

```
public interface IPProducto
{
    8 references
    string Nombre { get; }
    8 references
    decimal Precio { get; }
    4 references
    void ActualizarPrecio(decimal nuevoPrecio);
}
```

Modificación en Producto.cs

```
public class Producto : IPProducto
{
    ...
}
```

TiendaMocksTests.cs

```
using Xunit;
using Moq;

0 references
public class TiendaMocksTests
{
    [Fact]
    0 references
    public void AplicarDescuento_LlamaActualizarPrecioConValorCorrecto()
    {
        var mockProducto = new Mock<IProducto>();
        mockProducto.SetupGet(p => p.Precio).Returns(200);

        var tienda = new Tienda();
        tienda.AplicarDescuento(mockProducto.Object, 25);

        mockProducto.Verify(p => p.ActualizarPrecio(150), Times.Once);
    }
}
```

- Controladores y Resguardos:
 - Controladores: métodos que coordinan acciones, ej. AplicarDescuento.
 - Resguardos: validaciones o excepciones, ej. lanzar ArgumentNullException si el producto es null.
 - Test double: objeto que reemplaza a uno real para aislar la prueba.
 - Tipos: Mock (verifica llamadas), Stub (simula respuestas), Fake (implementación simplificada), Spy (registra interacciones).

4.

TiendaFixture.cs

```
public class TiendaFixture
{
    12 references
    public Tienda Tienda { get; private set; }

    0 references
    public TiendaFixture()
    {
        Tienda = new Tienda();
        Tienda.AgregarProducto(new Producto("Pan", 50, "Alimentos"));
        Tienda.AgregarProducto(new Producto("Leche", 100, "Lácteos"));
        Tienda.AgregarProducto(new Producto("Jugo", 120, "Bebidas"));
    }
}
```

TiendaFixtureTests.cs

```
using Xunit;

1 reference
public class TiendaFixtureTests : IClassFixture<TiendaFixture>
{
    4 references
    private readonly TiendaFixture _fixture;

    0 references
    public TiendaFixtureTests(TiendaFixture fixture)
    {
        _fixture = fixture;
    }

    [Fact]
    0 references
    public void BuscarProducto_Existente_UsandoFixture()
    {
        var producto = _fixture.Tienda.BuscarProducto("Leche");
        Assert.NotNull(producto);
        Assert.Equal("Leche", producto.Nombre);
    }

    [Fact]
    0 references
    public void AgregarProducto_UsandoFixture()
    {
        var nuevoProducto = new Producto("Queso", 80, "Lácteos");
        _fixture.Tienda.AgregarProducto(nuevoProducto);

        var buscado = _fixture.Tienda.BuscarProducto("Queso");
        Assert.NotNull(buscado);
        Assert.Equal(80, buscado.Precio);
    }
}
```

- Ventajas de fixtures:

- Evitan duplicar código de inicialización.
- Garantizan consistencia de datos entre pruebas.
- Facilitan mantenimiento y claridad del código de tests.

-Enfoque: caja negra, se prueba funcionalidad observable, no implementación interna.

-Setup y Teardown:

- Setup: preparar el entorno antes de cada prueba (ejm: inicializar tienda y productos).
- Teardown: limpiar recursos después de cada prueba (ejm: vaciar inventario, liberar memoria).

5.

Modificación en Tienda.cs

```
public decimal CalcularTotalCarrito(List<string> carrito)
{
    decimal total = 0;
    foreach (var nombre in carrito)
    {
        var producto = BuscarProducto(nombre);
        total += producto.Precio;
    }
    return total;
}
```

IntegracionTests.cs

```
using Xunit;

1 reference
public class IntegracionTests : IClassFixture<TiendaFixture>
{
    6 references
    private readonly TiendaFixture _fixture;

    0 references
    public IntegracionTests(TiendaFixture fixture)
    {
        _fixture = fixture;
    }

    [Fact]
    0 references
    public void CalcularTotalCarrito_AfterDescuentos()
    {
        var pan = _fixture.Tienda.BuscarProducto("Pan");
        var leche = _fixture.Tienda.BuscarProducto("Leche");
        _fixture.Tienda.AplicarDescuento(pan, 10);
        _fixture.Tienda.AplicarDescuento(leche, 20);

        var carrito = new List<string> { "Pan", "Leche" };
        decimal total = _fixture.Tienda.CalcularTotalCarrito(carrito);

        Assert.Equal(125, total);
    }
}
```

- Pruebas de integración ascendente:
 - Se comienza probando los componentes de forma individual y luego se integran progresivamente en flujos más completos.
- Ejemplo: primero se prueban de forma individual funcionalidades como ActualizarPrecio y AplicarDescuento.
- Luego se integran estas funcionalidades con Tienda y TiendaFixture para probar el flujo completo de aplicación de descuentos y cálculo del total del carrito mediante CalcularTotalCarrito.